

VagaLink

Encontre a vaga certa na hora certa

Documentação de Requisitos, Layout e Tecnologias Aplicadas

Disciplina: Programação Front End / Atividade Prática Final

Campo	Informação
Instituição	UniCesumar
Disciplina	Programação Front End
Professor	Esp. Dacio F. Machado
Tipo de entrega	Landing Page (HTML5, CSS3 e JavaScript)
Repositório de código	https://github.com/jpparralopes/Front-End_2026.git
Página publicada	https://jpparralopes.github.io/Front-End_2026/VagaLink/index.html
Equipe	João Pedro Parra Lopes, Enzo do Nascimento Lara, João Pedro Pissolato Zingra e Fernando Augusto Martinelli Filho

1. Introdução

Este documento apresenta a documentação técnica do projeto VagaLink, desenvolvido como Atividade Prática Final da disciplina de Programação Front End e ligada com a atividade de Mentalidade Criativa. O objetivo desta entrega é descrever, de forma similar a um documento de requisitos, os recursos implementados, as decisões de layout e interface e as tecnologias aplicadas na construção da página web do projeto.

O VagaLink é uma landing page que apresenta uma proposta de plataforma de reserva de vagas de estacionamento privadas. A página foi construída inteiramente com HTML5, CSS3 e JavaScript, sem o uso de frameworks ou bibliotecas externas, e tem como finalidade demonstrar o conceito do produto, suas funcionalidades principais e permitir uma simulação interativa do fluxo de busca e reserva de vagas.

1.1. Sobre o Projeto VagaLink

O VagaLink propõe-se a resolver um problema comum em centros urbanos: a dificuldade de encontrar vagas de estacionamento disponíveis. A plataforma conecta motoristas a vagas privadas (garagens e vagas residenciais ociosas), permitindo reserva antecipada por hora ou por dia e acesso à vaga por meio de QR Code.

Esta entrega corresponde à camada de apresentação (front-end) do projeto, conforme solicitado na atividade prática: uma Home Page / Landing Page funcional, publicável por meio de uma URL acessível, que demonstra o conceito, a proposta de valor e o fluxo de uso do produto.

2. Objetivos

2.1. Objetivo Geral

Desenvolver uma landing page completa, funcional e responsiva que apresente o produto VagaLink, comunique sua proposta de valor e demonstre, por meio de uma simulação client-side, o fluxo de busca e reserva de uma vaga de estacionamento.

2.2. Objetivos Específicos

- Apresentar o problema que a plataforma resolve e a solução proposta de forma clara e objetiva.
- Demonstrar o funcionamento do produto por meio de um passo a passo ("Como Funciona").
- Expor os principais benefícios e diferenciais da plataforma para o usuário final.
- Simular, com dados fictícios, a busca e a reserva de uma vaga de estacionamento.
- Reforçar a credibilidade do produto por meio de depoimentos de usuários.
- Disponibilizar um canal de contato (formulário) para dúvidas, parcerias e cadastro de vagas.
- Garantir uma interface visual moderna, coerente e responsiva para diferentes tamanhos de tela.

3. Tecnologias Aplicadas

Optou-se pelo uso de tecnologias front-end nativas (HTML5, CSS3 e JavaScript puro/"vanilla"), sem frameworks ou bibliotecas externas. Essa escolha foi feita por três motivos principais: (1) o escopo da página, uma landing page institucional, não demanda o gerenciamento de estado complexo de uma SPA; (2) a ausência de dependências externas simplifica a hospedagem em serviços de páginas estáticas, como o GitHub Pages; e (3) o uso das tecnologias base reforça o domínio dos fundamentos trabalhados na disciplina (estruturação semântica, estilização e manipulação do DOM).

Tecnologia	Aplicação no projeto
HTML5	Estruturação semântica do conteúdo (header, section, footer, form, nav), organizada em seções identificadas por id para navegação interna por âncoras.
CSS3	Estilização visual completa: layout com Flexbox e Grid, variações de cor, espaçamento, efeitos de hover, responsividade e identidade visual (tema escuro).
JavaScript (vanilla)	Manipulação do DOM, eventos de interface, geração dinâmica de elementos HTML (cards de vaga), validação de formulários e simulação de regras de negócio (busca e reserva).
Emojis / Ícones tipográficos	Utilizados como recurso visual leve (🔍 📍 ⭐ 🏠, entre outros), evitando a dependência de bibliotecas de ícones externas.
Hospedagem estática	Arquitetura compatível com serviços de hospedagem estática (ex.: GitHub Pages), por não depender de back-end ou build step.

4. Estrutura de Arquivos e Organização do Código

O projeto está organizado em três arquivos principais, seguindo a separação clássica de responsabilidades entre estrutura, apresentação e comportamento:

```
📁 vagalink/  
├── index.html    → Estrutura e conteúdo da página  
├── style.css     → Estilização visual (layout, cores, responsividade)  
├── script.js     → Comportamento dinâmico (busca, reserva, formulário)  
└── 📁 imagens/  → Logotipo e mockups utilizados na página  
    ├── VagaLink.png  
    ├── VagaLink_Iccon.png  
    └── vagalink-app-mockup.png
```

- **Separação de responsabilidades:** o HTML não contém estilos embutidos (inline) nem lógica de comportamento; toda a apresentação está em style.css e toda a interatividade em script.js.

- **Nomenclatura em português:** classes CSS, identificadores e variáveis JavaScript foram nomeados em português (ex.: cabeçalho, botao, vagas, reservar), mantendo a coerência com o público e o contexto do projeto.
- **Comentários no código:** o arquivo script.js utiliza comentários (// ---) para delimitar e identificar cada bloco funcional, facilitando a leitura e manutenção.

5. Arquitetura da Página (Seções)

A página é dividida em seções independentes, cada uma identificada por um id para permitir navegação interna via âncoras (menu fixo no topo). A tabela abaixo resume a função de cada seção, na ordem em que aparecem no documento.

Seção (id)	Descrição
header.cabecalho	Cabeçalho fixo (position: sticky) com logotipo, menu de navegação por âncoras e botão hamburguer para exibição em telas pequenas.
#hero	Seção de abertura (above the fold) com chamada principal, subtítulo, botões de ação ("Encontrar Vaga" e "Como funciona?"), estatísticas de destaque e um mockup ilustrativo do aplicativo.
#problema	Apresenta o problema de mercado (dificuldade para estacionar) por meio de quatro cartões com ícone, título e descrição.
#como-funciona	Demonstra o fluxo de uso em três passos numerados (Buscar → Reservar → Acessar com QR Code), conectados visualmente por setas.
#solucao	Detalha a proposta de solução, com uma imagem ilustrativa do aplicativo e uma grade de recursos-chave (localização, reserva instantânea, acesso seguro, avaliações).
#beneficios	Lista, em grade de duas colunas, os seis principais benefícios da plataforma para o usuário.
#busca	Simulação interativa de busca de vagas: formulário (cidade, bairro, data) e área de resultados gerada dinamicamente via JavaScript.
#avaliacoes	Exibe depoimentos fictícios de usuários, com nota, texto, avatar e identificação (nome, ocupação, cidade), reforçando prova social.
#contato	Apresenta informações de contato (e-mail, telefone, endereço) e um formulário para envio de mensagens, com validação e mensagem de sucesso.
footer.rodape	Rodapé com logotipo, descrição curta, links de navegação e ano vigente atualizado automaticamente via JavaScript.

6. Decisões de Layout e Interface

6.1. Identidade Visual e Paleta de Cores

Foi adotado um tema escuro (dark theme) como identidade visual da marca, transmitindo uma percepção tecnológica e moderna, alinhada a aplicativos de mobilidade e tecnologia urbana. A cor verde foi escolhida como cor de destaque (acento) por remeter à ideia de "disponibilidade" / "vaga livre" (analogia ao semáforo e às vagas de estacionamento sinalizadas em verde).

Cor	Código (HEX)	Uso
Fundo principal	#0d1117	Cor de fundo do body; base do tema escuro.
Fundo de seções alternadas / cartões	#161b22 / #1c2330	Diferenciação visual entre seções consecutivas e fundo dos cards.
Texto principal	#cdd9e5	Texto de leitura sobre o fundo escuro, com bom contraste e conforto visual.
Texto secundário	#8b949e	Subtítulos, descrições e textos de apoio, com menor ênfase.
Cor de destaque (primária)	#00c96e	Botões primários, ícones de destaque, preços, links ativos e bordas em hover, a "cor da marca".
Bordas	#2a3344	Delimitação sutil de cartões, divisores de seção e campos de formulário.
Alerta / categoria secundária	#ffbf00	Identifica vagas do tipo "aberta" (em contraste com vagas

6.2. Tipografia

A fonte adotada é a pilha tipográfica nativa do sistema (system-ui, sans-serif). Essa decisão prioriza desempenho (sem necessidade de carregar fontes externas), legibilidade e uma aparência consistente com o sistema operacional do usuário. A hierarquia visual é construída por meio de variação de tamanho e peso (font-weight): títulos de seção em destaque (700-800), textos de apoio em peso regular e com cor mais clara/suave.

6.3. Layout e Responsividade

O layout foi construído com Flexbox e CSS Grid, priorizando uma estrutura fluida que se adapta a diferentes tamanhos de tela:

- **Grades adaptativas:** uso de grid-template-columns: repeat(auto-fit, minmax(...)) nas seções de cartões (Problema, Benefícios, Avaliações), permitindo que o número de colunas se ajuste automaticamente à largura disponível, sem a necessidade de breakpoints manuais para cada componente.
- **Flexbox com quebra de linha:** seções como o Hero, "Como Funciona" e "Solução" utilizam display: flex com flex-wrap, permitindo que os blocos se reorganizem verticalmente em telas estreitas.
- **Cabeçalho fixo (sticky):** o menu principal permanece visível durante a rolagem (position: sticky), facilitando a navegação entre seções em páginas longas.
- **Botão de menu (hamburger):** previsto na estrutura HTML/CSS (.btn-menu) para exibição em telas pequenas no lugar do menu horizontal.
- **Rolagem suave:** a propriedade scroll-behavior: smooth foi aplicada ao elemento html, tornando a navegação por âncoras (ex.: "Início", "Como Funciona") mais fluida.

6.4. Componentes de Interface (UI Patterns)

- **Cartões (cards):** padrão reutilizado em praticamente todas as seções (problema, passos, benefícios, vagas, avaliações), com fundo levemente destacado, borda sutil e efeito de destaque da borda ao passar o mouse (hover), reforçando a sensação de interatividade.
- **Selos / etiquetas (badges):** pequenas etiquetas arredondadas usadas para indicar categorias (ex.: tipo de vaga "coberta"/"aberta", selo "Disponível" no mockup do Hero, rótulo de seção).
- **Mockup ilustrativo:** um "cartão de aplicativo" estilizado (simulando uma janela com barra de título, em referência ao padrão visual de sistemas macOS) é utilizado no Hero para representar visualmente o produto antes mesmo de qualquer funcionalidade real ser acionada.
- **Formulários padronizados:** campos de busca e de contato compartilham o mesmo padrão visual (.campo), com rótulo em maiúsculas, campo com fundo escuro e destaque de borda verde no foco (:focus), reforçando consistência visual em toda a aplicação.

7. Funcionalidades Implementadas em JavaScript

O arquivo script.js concentra toda a lógica de interação da página. Como se trata de uma landing page de demonstração, as regras de negócio (busca e reserva de vagas) são simuladas no lado do cliente, com dados fictícios (mock data), sem comunicação com um servidor ou banco de dados real.

7.1. Atualização automática do ano no rodapé

O ano exibido no rodapé ("© 2026 VagaLink") é preenchido dinamicamente a partir da data do sistema do usuário, evitando a necessidade de atualização manual a cada ano:

```
document.getElementById('anoAtual').textContent = new Date().getFullYear();
```

7.2. Simulação de busca de vagas

Um array de objetos (vagas) armazena cinco vagas fictícias, cada uma com nome, endereço, preço, tipo (coberta/aberta) e avaliação. Ao clicar no botão "Buscar Vagas":

- o script valida se os campos cidade, bairro e data foram preenchidos; caso contrário, exibe uma mensagem de alerta na própria área de resultados;
- se todos os campos estiverem preenchidos, a função `montarCardVaga()` converte cada item do array em um bloco HTML (card), utilizando template strings;
- os cards gerados são inseridos dinamicamente na área de resultados (`#resultados`) por meio de `innerHTML`, organizados em uma grade (`.grade-resultados`).

Observação: a busca atual não filtra as vagas de acordo com os valores informados (cidade/bairro/data), todas as cinco vagas fictícias são sempre exibidas quando a validação é aprovada. Trata-se de uma simulação do fluxo de interface, e não de uma busca real.

7.3. Simulação de reserva

Cada card de vaga possui um botão "Reservar", associado à função `reservar(botao)`. Ao ser clicado, o próprio botão é substituído (`outerHTML`) por uma mensagem de confirmação ("✓ Reserva confirmada! Seu QR Code será enviado por e-mail."), simulando o desfecho do fluxo apresentado na seção "Como Funciona", sem persistência de dados ou envio real de e-mail/QR Code.

7.4. Validação e envio do formulário de contato

O envio padrão do formulário é interceptado (`e.preventDefault()`) para que o comportamento seja tratado via JavaScript:

- os campos nome, e-mail e mensagem são lidos e têm espaços nas extremidades removidos (`trim()`);
- caso algum campo esteja vazio, é exibido um `alert()` solicitando o preenchimento completo;
- se todos os campos estiverem preenchidos, a mensagem de sucesso (`.msg-sucesso`) é exibida e o formulário é limpo (`reset()`).

Assim como na busca de vagas, o envio do formulário é tratado apenas no front-end: nenhuma requisição é enviada a um servidor, e-mail ou serviço externo nesta versão da página.

8. Boas Práticas Adotadas

- **HTML semântico:** uso de elementos com significado estrutural (`header`, `nav`, `section`, `footer`, `form`) em vez de `div`s genéricas, favorecendo legibilidade e acessibilidade.
- **Separação de responsabilidades:** estrutura (HTML), apresentação (CSS) e comportamento (JavaScript) mantidos em arquivos distintos, sem estilos ou scripts inline.
- **Reuso de componentes visuais:** classes utilitárias e padrões de cartão (`.card`) reaproveitados em diversas seções, reduzindo duplicação de CSS.
- **Atributos de acessibilidade:** uso de `aria-label` no botão de menu e de texto alternativo (`alt`) em imagens relevantes.
- **Validação de formulários:** uso do atributo HTML `required` combinado com validação adicional em JavaScript antes do processamento dos dados.
- **Código comentado e organizado:** blocos de funcionalidade no `script.js` são delimitados por comentários, facilitando manutenção futura.

- **Compatibilidade com hospedagem estática:** ausência de dependências de back-end, possibilitando publicação direta em serviços como GitHub Pages.

9. Limitações Atuais e Melhorias Futuras

Por se tratar de uma landing page de demonstração, alguns pontos foram identificados como limitações conhecidas e oportunidades de evolução do projeto:

- **Menu mobile (hamburger):** o botão (.btn-menu) já está estruturado em HTML e estilizado em CSS, mas o script de abertura/fechamento do menu em telas pequenas ainda não foi implementado em script.js.
- **Busca sem filtro real:** a busca de vagas não considera, de fato, os valores de cidade, bairro e data informados pelo usuário; todas as vagas fictícias são exibidas indistintamente.
- **Persistência de dados:** reservas e mensagens de contato não são enviadas a nenhum servidor, banco de dados ou serviço de e-mail, todo o comportamento ocorre apenas no navegador (client-side).
- **Integração futura sugerida:** conexão com uma API real de vagas, autenticação de usuários, back-end para persistência de reservas e geração de QR Codes reais, evoluindo a aplicação de uma landing page demonstrativa para uma plataforma funcional completa.

10. Considerações Finais

O projeto VagaLink cumpre o objetivo proposto na atividade prática da disciplina de Programação Front End: apresentar, por meio de uma página web funcional, responsiva e visualmente coerente, o conceito de um produto digital, suas funcionalidades centrais e seu fluxo de uso, utilizando exclusivamente HTML5, CSS3 e JavaScript.

A escolha por tecnologias nativas, aliada a boas práticas de organização de código e a decisões de design fundamentadas (paleta de cores, tipografia, componentes reutilizáveis e responsividade), resultou em uma entrega coesa, de fácil manutenção e compatível com qualquer serviço de hospedagem de páginas estáticas.